
SCC Kernel Fortress: Executable Evaluation Gates for Runtime Compliance in Agentic Systems

Ian Farquharson

Abstract

Stateful agentic systems operate through tool-use loops, memory updates, external calls, and audit traces, yet existing mechanisms do not reject invalid runtime transitions before state acceptance. We present SCC KERNEL FORTRESS, an executable evaluation framework that implements a deterministic pre-acceptance checker over a previous state, candidate next state, canonical event, audit event, proof-obligation bundle, and execution descriptor. A transition is accepted only when its evidence satisfies the implemented SCC obligations: canonical encoding agreement, audit-chain consistency, protected-coordinate isolation, governance-state validity, risk-to-halt enforcement, halt absorption, lineage continuation, and normalized Exact/Stutter/SafeRefined step classification. The release includes a Rust source-of-truth checker, TypeScript reference checker, golden vectors with binary canonical bytes, a 14-case negative corpus with named first-failure gates, property-test scaffolding, fuzz targets, a Lean/Coq formal bridge for the implemented v1 subset, federation evidence fixtures, numerical-stability bounds, a trusted computing base ledger, and CI release gates. SCC KERNEL FORTRESS does not claim general AI safety, full SCC theorem mechanization, or compiler, operating-system, or cryptographic correctness. As an Evaluations & Datasets artifact, it contributes a reusable protocol and fixture corpus for testing transition-legality claims in stateful agentic runtimes. SCC KERNEL FORTRESS is the executable boundary artifact; the Full SCC Canon is the formal authority it points back to.

1 Introduction

Modern AI systems are moving from stateless prediction toward stateful execution. They call tools, write files, update memory, route tasks, invoke policies, generate audit records, and sometimes attempt recovery after failure. In these settings, safety cannot be evaluated only by inspecting the final output. The critical question becomes whether the system’s *runtime evolution* was lawful, auditable, and recoverable under the rules governing the runtime.

A stateful system can fail even when its visible output looks acceptable. It may erase lineage, mutate protected memory, lower a failure mode without authorization, bypass a halt condition, corrupt an audit chain, or accept a next state without machine-checkable evidence. These are not ordinary output-quality failures. They are transition-validity failures.

Most monitoring mechanisms are not designed to reject invalid transitions before acceptance. Logs can record what happened, but they do not necessarily prevent illegal state change. Prompts can declare policy, but they do not mechanically enforce legality. Schemas can validate object shape, but they do not verify audit continuity, protected-state preservation, governance bounds, risk escalation, halt absorption, or refinement correctness. Post-hoc audits can detect violations after the fact, but by then the invalid state may already have influenced later execution.

SCC KERNEL FORTRESS addresses this gap by treating runtime compliance as a pre-acceptance evaluation problem. Before a candidate next state can enter the accepted trajectory, it must pass a

Table 1: SCC terminology as operational runtime concepts.

Internal term	Operational phrase
Proof-obligation bundle	Machine-checkable transition evidence
Constitutional admissibility	Runtime compliance rule satisfaction
Protected-coordinate isolation	Safety-critical state immutability
Halt absorption	Safety stop cannot be bypassed
Governance simplex	Valid bounded governance weights
Lineage continuation	Execution history continuity
SafeRefined	Risk-nonincreasing safe refinement

39 deterministic checker. If the proof-obligation bundle is absent, malformed, hash-inconsistent, audit-
 40 inconsistent, lineage-breaking, risk-invalid, halt-invalid, governance-invalid, or refinement-invalid,
 41 the transition is rejected.

42 This paper presents SCC KERNEL FORTRESS as the executable front layer of Synthetic Cognitive
 43 Coding, or SCC. The Full SCC Canon defines the broader constitutional-operational theory: typed
 44 state spaces, admissible trajectories, proof obligations, governance rules, risk and halt laws, memory
 45 and audit constraints, refinement semantics, implementation correspondence, distributed coherence,
 46 mechanization targets, and assumption boundaries [3]. The Engineering Handbook translates that
 47 canon into builder-facing discipline: implement the kernel first, keep it small, treat proof-obligation
 48 bundles as central artifacts, keep the checker deterministic, separate product logic from compliance
 49 logic, harden the trusted computing base, and reject ordinary execution when the bundle fails [4].
 50 SCC KERNEL FORTRESS sits between them: it demonstrates how selected canonical obligations
 51 become deterministic runtime gates.

52 **Artifact Claim.** Given a previous state, candidate next state, canonical event, audit event,
 proof-obligation bundle, and execution descriptor, SCC KERNEL FORTRESS deterministically
 accepts only transitions satisfying the implemented SCC obligations and rejects named violation
 classes before state acceptance.

53 **Non-Claim.** SCC KERNEL FORTRESS does not prove general AI safety, full SCC theorem
 mechanization, compiler correctness, operating-system correctness, cryptographic soundness
 of SHA-256, or correctness of every possible SCC implementation.

54 **Why this is an E&D artifact.** The contribution is not a model-quality leaderboard or a new
 55 training recipe. It is a reusable evaluation protocol for a failure mode that ordinary output evaluation
 56 underspecifies: whether a stateful agent runtime can make a candidate transition legally admissible
 57 before the transition is committed. The artifact defines the evaluation unit, the accept/reject predicate,
 58 positive controls, negative controls, fixture-level first-failure expectations, and release gates. Other
 59 researchers can use the package in three ways: run the supplied checker against the committed corpus,
 60 implement an independent checker and compare first-failure behavior, or adapt the fixture protocol to
 61 audit their own stateful agent loops.

62 2 Operational terminology

63 The kernel preserves formal SCC vocabulary, but the evaluation artifact introduces each term through
 64 the operational problem it solves. Table 1 gives the translation used throughout this paper.

65 3 Evaluation protocol

66 SCC KERNEL FORTRESS is submitted as an evaluation artifact because its primary contribution
 67 is not a new model, dataset, or training method. Its contribution is a reproducible mechanism for
 68 evaluating whether runtime-compliance claims are enforced by deterministic acceptance logic. The
 69 evaluation question is simple: given a claimed compliant transition, does the runtime provide enough
 70 machine-checkable evidence for that transition to be accepted?

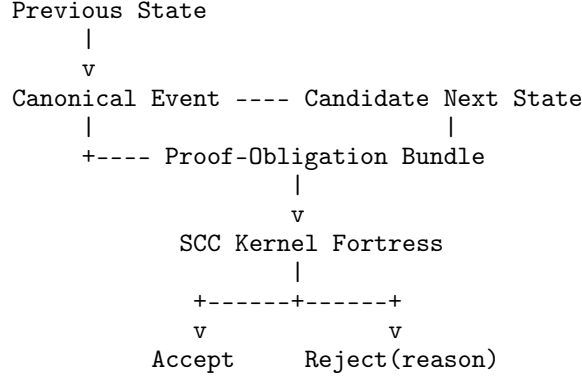


Figure 1: SCC KERNEL FORTRESS evaluates runtime compliance before state acceptance. A candidate transition enters the accepted trajectory only if its proof-obligation bundle, canonical hashes, audit event, risk/halt behavior, lineage, and refinement classification satisfy the implemented obligations.

Table 2: Evaluation protocol.

Protocol element	Definition
Evaluation unit	One candidate state transition.
Inputs	Previous state, candidate next state, canonical event, audit event, proof-obligation bundle, and execution descriptor.
Decision	Accept, or reject with a named reason.
Positive controls	Golden vectors for Exact, Stutter, and SafeRefined steps.
Negative controls	Named first-failure corpus for malformed or unlawful bundles.
Reproducibility	Rust/TypeScript checks, committed binary vectors, forbidden-import checks, and CI gates.

71 The evaluation unit is one candidate runtime transition. The checker receives six inputs and returns
72 either acceptance or a named rejection. This is not a probability, policy score, or advisory warning; it
73 is a deterministic acceptance predicate.

74 4 Case study: runtime compliance failures in a stateful agent loop

75 A simple stateful agent loop can be represented as

$$State_t \rightarrow AgentEvent \rightarrow CandidateState_{t+1} \rightarrow POBundle \rightarrow Checker \rightarrow Accept/Reject. \quad (1)$$

76 SCC KERNEL FORTRESS evaluates whether the candidate transition is admissible before it becomes
77 part of runtime history. Table 3 maps representative agent-loop compliance failures to committed
78 negative-corpus fixtures. Each fixture is expected to reject at the named first-failure gate; a different
79 first-failure gate is treated as a regression.

80 The point of the case study is not that SCC KERNEL FORTRESS solves every agent-safety problem.
81 The point is that it catches a class of runtime-compliance failures that ordinary output evaluation can
82 miss.

83 5 Baseline comparison

84 SCC KERNEL FORTRESS is best understood by contrast with mechanisms that record, advise, or
85 partially validate behavior but do not necessarily reject invalid transitions before state acceptance.
86 Table 4 is a qualitative capability comparison. It does not claim that all systems in a class lack
87 stronger variants; it identifies which guarantees are provided by this artifact as packaged and tested.

88 The differentiator is pre-acceptance enforcement. SCC KERNEL FORTRESS does not merely record
89 that a transition occurred; it determines whether the transition may be accepted.

Table 3: Representative agent-loop failures mapped to committed negative-corpus fixtures.

Failure scenario	Runtime symptom	Committed fixture	First-failure gate
Missing evidence	Agent proposes a state update without a valid proof-obligation bundle.	<code>missing_required_field.json</code>	<code>missing_required_field</code>
Audit corruption	Runtime rewrites or mismatches the audit hash.	<code>audit_event_hash_bad.json</code>	<code>audit_event_hash_bad</code>
Risk escalation	Risk threshold is crossed but the system does not halt.	<code>risk_threshold_without_halt.json</code>	<code>risk_threshold_without_halt</code>
Halt escape	System exits halt without authorized recovery.	<code>halt_absorption_violation.json</code>	<code>halt_absorption_violation</code>
Governance corruption	Governance weights leave the valid simplex.	<code>invalid_simplex.json</code>	<code>invalid_simplex</code>
Lineage erasure	Runtime breaks or deletes continuation history.	<code>lineage_erasure.json</code>	<code>lineage_erasure</code>
Protected-state mutation	Ordinary execution mutates protected coordinates.	<code>protected_mutation.json</code>	<code>protected_mutation</code>
Refinement misclassification	Bundle declares a mode inconsistent with the checker classification.	<code>wrong_step_mode.json</code>	<code>wrong_step_mode</code>

Table 4: Pre-acceptance enforcement compared with common monitoring mechanisms.

Mechanism	Records	Pre-accept reject	Checks hash/audit	Enforces halt/risk	Protects state
Plain logs	Yes	No	No	No	No
Prompt policy	Weakly	No	No	No	No
JSON schema validation	Shape only	Partially	No	No	No
Post-hoc audit	Yes	No	Sometimes	No	No
Runtime policy wrapper	Sometimes	Sometimes	Rarely	Sometimes	Rarely
SCC KERNEL FORTRESS	Yes	Yes	Yes	Yes	Yes

90 6 Implemented obligations and checker pipeline

91 The checker is a predicate over a previous state s , next state s' , event e , audit event a , proof-obligation
 92 bundle p , and execution descriptor Θ :

$$\text{accepts}(p, s, s', e, a, \Theta) \in \{\text{Ok}, \text{Err}(r)\}. \quad (2)$$

93 It is pure: no wall-clock time, randomness, network calls, environment variables, mutable globals,
 94 telemetry, or product runtime decisions participate in the decision.

Listing 1: Rust source-of-truth checker public signature.

```

95 pub fn accepts(
96     po: &POBundle, prev: &SCCState, next: &SCCState,
97     event: &CanonicalEvent, audit_event: &AuditEvent,
98     env: &ExecutionEnv,
99 ) -> Result<(), CheckerError> { ... }
```

100 The Rust `accepts` function is the single gate. It rejects in this order: schema mismatch, checker-
 101 version mismatch, invalid mode, missing required PO field, canonical hash mismatch, domain
 102 violation, protected-coordinate mutation, halt violation, governance violation, risk violation, lineage
 103 violation, audit-chain break, and refinement violation. The order is intentional because negative
 104 corpus cases assert the first failing gate.

Table 5: Canon-to-executable mapping.

SCC source	Executable obligation	Negative/golden gate
Article F9, Article C11	Valid proof-obligation bundle or no lawful step.	<code>missing_required_field</code>
Article 29	Governance vector must remain in the simplex.	<code>invalid_simplex</code>
Article 30	Threshold risk must force halt.	<code>risk_threshold_without_halt</code>
Article 32	Halt is absorbing unless authorized recovery applies.	<code>halt_absorption_violation</code>
Article 35	Audit chain is deterministic and hash-bound.	<code>audit_event_hash_bad</code>
Article 36	Lineage must continue.	<code>lineage_erasure</code>
Articles 52–53	Step mode must be <code>Exact</code> , <code>Stutter</code> , or <code>SafeRefined</code> .	Three golden modes; <code>wrong_step_mode</code>
Articles 73, 80	Trusted computing base and external assumptions must be explicit.	<code>tcb_ledger.md</code>

105 The v1 kernel defines typed records for `SCCState`, `CanonicalEvent`, `AuditEvent`, `POBundle`, and
 106 `ExecutionEnv`. All records are serialized by domain-separated binary encoders. Strings are UTF-8
 107 with u32 lengths, integers are big-endian, maps are key-sorted, and floating point values are finite
 108 IEEE-754 binary64 in big-endian order with negative zero normalized. JSON is a carrier format only;
 109 the committed `.bin` vectors are the byte-level contract. The audit chain avoids circularity by hashing
 110 the next state with the audit coordinate zeroed:

$$H_a = \text{SHA256}(\text{AuditEvent}(H_t, H(e), H(s_t), H_{\text{body}}(s_{t+1}), \Theta)). \quad (3)$$

111 The checker then requires both $p.\text{audit_event_hash} = H_a$ and $s_{t+1}.\text{audit_hash} = H_a$.

112 7 Step classification, governance, risk, and halt

113 The canon separates full abstraction ρ from normalized simulation projection $\hat{\rho}$ so that audit and
 114 lineage can advance during behavioral stuttering [3]. The implementation follows that split. The
 115 normalized projection drops administrative audit and lineage coordinates while retaining compute
 116 root, memory root, vector clock, governance weights, failure mode, risk vector, and protected root.

117 Given the induced exact successor $\hat{T}(\hat{\rho}(s), e)$, the classifier returns exactly one of:

$$\text{Exact} : \hat{\rho}(s') = \hat{T}(\hat{\rho}(s), e), \quad (4)$$

$$\text{Stutter} : \hat{\rho}(s') = \hat{\rho}(s), \quad (5)$$

$$\text{SafeRefined} : \hat{\rho}(s') \preceq_{\text{safe}} \hat{T}(\hat{\rho}(s), e). \quad (6)$$

118 The v1 safety preorder is defined coordinatewise:

$$s' \preceq_{\text{safe}} t \iff \text{coord}(s') = \text{coord}(t) \wedge \sum_i r_i(s') \leq \sum_i r_i(t) \wedge \text{mode}(s') \geq \text{mode}(t), \quad (7)$$

119 where `coord` denotes the structural coordinates (compute root, memory root, vector clock, governance
 120 weights, protected root), r_i are risk-vector components, and `mode` is failure-mode severity (higher is
 121 more severe). A bundle whose declared mode disagrees with the classifier is rejected.

122 Governance weights must lie in the probability simplex. The transition helper uses the standard
 123 sort-threshold-renormalize projection family for simplex or ℓ_1 -ball constraints [6, 19]. The checker
 124 does not trust a projection witness blindly: it checks that the next vector lies in the simplex and that
 125 L2 drift is within the descriptor bound.

126 Risk is executable law in v1. If $\sum_i r_i \geq \Theta.\text{risk_halt_threshold}$, then the next failure mode must be
 127 3. If the previous failure mode is 3 and the bundle does not invoke an authorized recovery path, the
 128 next failure mode must remain 3. Ordinary execution also cannot silently lower severity.

129 8 Reproducibility and release gates

130 The author-identified supplementary package contains three accepted golden modes (`Exact`, `Stutter`,
 131 and `SafeRefined`), fourteen negative cases, JSON and binary vector forms, Rust tests, TypeScript tests,
 132 property-test scaffolding, fuzz targets, schemas, a TCB ledger, and a CI workflow. The TypeScript

reference suite contains 22 tests: three golden acceptance tests, fourteen negative rejection tests, one replay determinism test, and four stress tests covering memory-compression lineage, repaired-hash lineage rejection, simplex numerical stability, and federation quorum arithmetic. The release gate is:

Listing 2: Release gate used by local and CI validation.

```
python tools/validate_vectors.py
python tools/forbidden_imports.py
cd rust && cargo test --all-features
cd rust && cargo test --test negative_corpus
cd rust && cargo test --test golden_vectors
cd rust && cargo test --test proptest_invariants
cd rust && cargo test --test memory_lineage_compression
cd rust && cargo test --test numerical_stability
cd rust && cargo test --test federation_quorum
cd ts && npm test
python tools/validate_formal_artifacts.py
bash scripts/run_formal_gates.sh
```

The submitted artifact should be considered valid only when these commands, or the included CI workflow that executes the same checks on a Rust- and Node-equipped runner, complete successfully. The artifact also includes a local packaging transcript for vector validation, forbidden-import scanning, and the TypeScript reference suite. We do not use unobserved local commands as evidence; unsupported release claims must be backed by the supplied CI or by an independently reproduced local transcript.

Table 6: Evidence tiers for reproducing the artifact claim.

Tier	Reviewer action	Evidence produced
Static	Inspect README, schemas, TCB ledger, fixture manifests, and checker pipeline.	Confirms artifact scope, inputs, claims, and trust boundary.
Vector	Run <code>validate_vectors.py</code> and inspect committed <code>.bin</code> fixtures.	Confirms byte-level fixture consistency for golden and negative cases.
Reference	Run the TypeScript suite under Node 22+.	Confirms 22 deterministic reference checks over golden, negative, replay, compression-lineage, numerical, and federation arithmetic cases.
Authority	Run Rust release gates under a Rust-equipped runner or the included CI workflow.	Confirms source-of-truth checker behavior, negative-corpus first-failure gates, golden-vector acceptance, and property tests.

9 Artifact use cases

SCC KERNEL FORTRESS is packaged for use as an executable evaluation harness, not only as a reference implementation. Table 7 summarizes the intended reviewer and downstream use cases. Each use case is intentionally transition-level: the artifact evaluates a candidate state transition and its evidence, not the natural-language quality of a generated answer.

10 Mechanization, stress, federation, and numerical hardening

This release turns the prior mechanization roadmap into a concrete bridge for the implemented v1 subset. The supplement includes a Lean 4 [1] micro-model of the gate, a seven-stage witness formalization, and a Coq [2] proof fragment for required-bundle reflection and quorum-intersection arithmetic. The mechanized claim is intentionally scoped: in the small formal model, acceptance implies Prop-level v1 obligations such as required bundle fields, protected-coordinate preservation, halt absorption, risk-to-halt implication, lineage continuation, and stage-witness completeness. The bridge does not prove the entire SCC canon, compiler correctness, SHA-256 security, or operating-system correctness.

These additions strengthen the artifact claim without enlarging it. The single-node checker remains the core evaluation unit. Mechanization proves the boundary in a small formal model; Rust/TypeScript stress tests increase executable coverage; federation fixtures specify distributed evidence that

Table 7: Executable artifact use cases.

Use case	What the evaluator does	Required evidence
Submission inspection	Run the release gate over the supplied package.	Golden modes accept; all negative fixtures reject at named first-failure gates.
Runtime regression test	Add the fixture corpus to a stateful agent runtime’s CI.	No transition can commit unless the checker accepts its proof-obligation bundle.
Independent checker comparison	Reimplement the acceptance predicate in another language.	Decisions and first-failure gates match the committed manifests.
Policy-wrapper audit	Feed proposed state transitions through SCC KERNEL FORTRESS before logging or committing them.	Wrapper cannot substitute logs, prompts, or schemas for machine-checkable transition evidence.
Mechanization target	Translate the Rust gate and fixture obligations into proof-assistant lemmas.	The TCB ledger and canon mapping define what must be proven versus assumed.

Table 8: Hardening additions beyond the single-node fixture corpus.

Area	Added evidence	Reviewer handle
Mechanization	Lean Bool-to-Prop gate soundness, Lean seven-stage witness bridge, Coq required-bundle and quorum arithmetic proofs.	<code>formal/</code> , <code>mechanization_manifest.json</code>
Rust stress	Added memory-lineage compression, numerical-stability, and federation-quorum integration tests, plus a lineage fuzz target.	<code>rust/tests/*</code> , <code>rust/fuzz/*</code>
Memory/lineage	Compression stress repairs hashes first, then tests accepted lineage continuation and semantic lineage-erasure rejection.	<code>memory_lineage_compression.rs</code>
Federation	Distributed story is a certificate layer over single-node legality: quorum certificates, cross-shard lineage binding, and bounded halt propagation.	<code>federation/manifest.json</code>
Numerics	TCB ledger now specifies finite binary64 encoding, negative-zero normalization, simplex tolerance, drift bounds, and near-threshold risk discipline.	<code>docs/numerical_stability_tcb.md</code>
Navigation	One-page system map reduces doc sprawl and connects Canon, Handbook, Kernel Fortress, and Builder Scaffold.	<code>docs/SCC_SYSTEM_OVERVIEW_ONE_PAGER.md</code>

171 composes with, rather than replaces, the single-node gate; numerical rules make floating-point trust
 172 explicit in the TCB ledger.

173 11 Threat model, limitations, and canon bridge

174 SCC KERNEL FORTRESS targets runtime-compliance failures in which a stateful system attempts
 175 to accept a transition that lacks required machine-checkable evidence, corrupts canonical hashes
 176 or audit continuity, mutates protected coordinates through ordinary execution, erases or severs
 177 lineage, crosses a risk threshold without entering halt, exits halt without authorized recovery, submits
 178 invalid governance-state updates, misdeclares its refinement mode, or depends on nondeterministic or
 179 unauthorized runtime inputs. This is an ML-evaluation contribution at the runtime layer: it evaluates
 180 transition legality in stateful agentic systems, rather than model likelihood, task score, or response
 181 preference.

182 It does not defend against every possible system compromise. It does not prove compiler correctness,
 183 operating-system correctness, hardware correctness, cryptographic primitive security, model-objective
 184 safety, or semantic harmlessness of generated content. It does not evaluate natural-language helpful-
 185 ness, truthfulness, toxicity, bias, or preference alignment. The implemented v1 subset now includes a
 186 formal bridge and proof targets, but the artifact still does not claim full mechanical proof of the entire

187 SCC canon or of the surrounding compiler, operating system, cryptographic primitive, hardware, or
 188 deployment substrate. These limits are not footnotes; they are part of the artifact’s trust boundary.

189 SCC KERNEL FORTRESS is not the whole SCC system. It is the executable gate. The Full SCC
 190 Canon provides the broader formal authority: state spaces, constitutional clauses, admissible domains,
 191 transition laws, proof-obligation requirements, governance and risk dynamics, audit determinism,
 192 lineage continuation, refinement semantics, implementation correspondence, distributed coherence,
 193 mechanization targets, and assumption boundaries [3]. The Engineering Handbook translates that
 194 authority into builder-facing instructions for a small deterministic kernel, proof-obligation bundles,
 195 TCB hardening, replay discipline, and release gates [4]. The executable artifact makes compliance
 196 testable; the canon supplies the constitutional-operational law behind the tests.

197 12 Related work

198 SCC KERNEL FORTRESS is adjacent to runtime verification, where monitors check properties of
 199 executions [9, 8], and to proof-carrying code, where consumers check explicit evidence before
 200 accepting code [13]. Reference-monitor and security-policy work motivate a mediating enforcement
 201 point for security-relevant decisions [5, 18]. Policy-as-code systems such as Open Policy Agent/Rego
 202 decouple policy decisions from application code and express policies over structured data [14]. Secure
 203 audit-log work studies tamper-evident logging after events occur [17]. SCC KERNEL FORTRESS
 204 is complementary: it makes the transition itself carry typed evidence, canonical byte commitments,
 205 risk/halt obligations, lineage constraints, and named rejection gates before acceptance.

206 Agent and tool-use evaluations such as AgentBench, ToolLLM/ToolBench, ToolSandbox, and
 207 ToolEmu evaluate interactive task performance, tool-use capability, or risk discovery in tool-rich
 208 environments [10, 15, 11, 16]. ML safety evaluations and red-teaming frameworks study harmful
 209 outputs, refusal robustness, and adversarial prompt discovery [7, 12]. SCC KERNEL FORTRESS
 210 occupies a different evaluation layer: it does not score the semantic quality of an agent’s response,
 211 but checks whether the runtime transition that produced or followed that response was admissible
 212 before state acceptance.

213 13 Conclusion

214 SCC KERNEL FORTRESS moves SCC enforcement from advisory documentation into an executable
 215 proof-of-boundary. It evaluates one candidate state transition at a time, requires machine-checkable
 216 evidence, validates canonical hashes and audit continuity, enforces protected-state, governance, risk,
 217 halt, lineage, and refinement obligations, and rejects named failure classes before acceptance.

218 The bridge to the Full SCC Canon is the central contribution. SCC KERNEL FORTRESS shows
 219 that selected canonical obligations can become machinery. The canon explains the law behind that
 220 machinery. Together, they make runtime compliance reviewable, reproducible, and falsifiable: A
 221 system that cannot prove the legality of its next step has no right to take it.

222 References

- 223 [1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming
 224 language. In *Proceedings of the 28th International Conference on Automated Deduction (CADE-*
 225 *28)*, pages 625–635, 2021.
- 226 [2] The Coq Development Team. *The Coq Proof Assistant, version 8.19*. Zenodo, 2024. <https://doi.org/10.5281/zenodo.10616045>
 227
- 228 [3] Ian Farquharson. *Synthetic Cognitive Coding (SCC): Formal Constitutional-Operational Speci-*
 229 *fication, Final Canon, Scholarly Edition*. Author-identified supplemental manuscript, 2026.
- 230 [4] Ian Farquharson. *Synthetic Cognitive Coding (SCC): Engineering Handbook, Hardened Builder*
 231 *Edition*. Author-identified supplemental manuscript, 2026.
- 232 [5] James P. Anderson. *Computer Security Technology Planning Study*. Technical Report ESD-TR-
 233 73-51, 1972.

- 234 [6] John C. Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections
235 onto the ℓ_1 -ball for learning in high dimensions. In *Proceedings of the 25th International*
236 *Conference on Machine Learning*, pages 272–279, 2008.
- 237 [7] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath,
238 Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna
239 Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El-Showk, Stanislav
240 Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Josh Jacobson,
241 Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Eli Tran-Johnson, Dario Amodei,
242 Tom Brown, Nicholas Joseph, Sam McCandlish, Chris Olah, Jared Kaplan, and Jack Clark. Red
243 teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned.
244 arXiv:2209.07858, 2022.
- 245 [8] Klaus Havelund and Grigore Rosu. Efficient monitoring of safety properties. *International*
246 *Journal on Software Tools for Technology Transfer*, 6(2):158–173, 2004.
- 247 [9] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of*
248 *Logic and Algebraic Programming*, 78(5):293–303, 2009.
- 249 [10] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,
250 Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui
251 Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie
252 Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning*
253 *Representations*, 2024.
- 254 [11] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma,
255 Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, con-
256 versational, interactive evaluation benchmark for LLM tool use capabilities. arXiv:2408.04682,
257 2025.
- 258 [12] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham
259 Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A
260 standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings*
261 *of the 41st International Conference on Machine Learning*, 2024.
- 262 [13] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT*
263 *Symposium on Principles of Programming Languages*, pages 106–119, 1997.
- 264 [14] Open Policy Agent Authors. Open Policy Agent documentation: Policy language and Rego.
265 2026. URL: <https://www.openpolicyagent.org/docs/policy-language>.
- 266 [15] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong,
267 Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou,
268 Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large lan-
269 guage models to master 16000+ real-world APIs. In *International Conference on Learning*
270 *Representations*, 2024.
- 271 [16] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann
272 Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with
273 an LM-emulated sandbox. arXiv:2309.15817, 2023.
- 274 [17] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Trans-*
275 *actions on Information and System Security*, 2(2):159–176, 1999.
- 276 [18] Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System*
277 *Security*, 3(1):30–50, 2000.
- 278 [19] Weiran Wang and Miguel A. Carreira-Perpinan. Projection onto the probability simplex: An
279 efficient algorithm with a simple proof, and an application. arXiv:1309.1541, 2013.

280 A Supplementary reproducibility notes

281 The release package is intended to be evaluated as an executable artifact. The author-identified
282 archive should contain the Rust crate, TypeScript reference checker, schemas, golden vectors, binary
283 canonical vectors, negative corpus, TCB ledger, validation tools, Makefile, and CI workflow. The
284 negative-corpus manifest contains fourteen cases: `audit_event_hash_bad`, `checker_version_`
285 `mismatch`, `descriptor_mismatch_audit`, `event_hash_bad`, `halt_absorption_violation`,
286 `invalid_simplex`, `lineage_erasure`, `missing_required_field`, `next_hash_bad`, `prev_`
287 `hash_bad`, `protected_mutation`, `risk_threshold_without_halt`, `schema_mismatch`, and
288 `wrong_step_mode`. Each case is committed as a JSON fixture and checked against its expected
289 first-failure gate.

290 B Supplementary mechanization and distributed notes

291 The supplement also contains `formal/lean`, `formal/coq`, `mechanization_manifest.json`,
292 `federation/`, `docs/numerical_stability_tcb.md`, and `docs/SCC_SYSTEM_OVERVIEW_ONE_`
293 `PAGER.md`. The formal files are part of the evidence package; proof-assistant execution must be
294 reported from a proof-equipped runner and must not be inferred from the static source tree alone.

295 **Reproducibility Statement.** The artifact is fully reproducible from the public repository at `https:`
296 `//github.com/IceMasterT/SCC-Formal-Constitution`. The release gate is:

```
297 python tools/validate_vectors.py
298 python tools/forbidden_imports.py
299 cd rust && cargo test --all-features
300 cd ts && npm test
301 bash scripts/run_formal_gates.sh
```

302 The repository includes committed binary canonical vectors (`.bin`), a Docker image, and a CI
303 workflow that executes the same commands on a Rust- and Node-equipped runner. All code is
304 released under Apache 2.0; paper and fixture corpus under CC BY 4.0; canonical documents under
305 CC BY-SA 4.0.